

Relay: a Slack-like computer-use environment

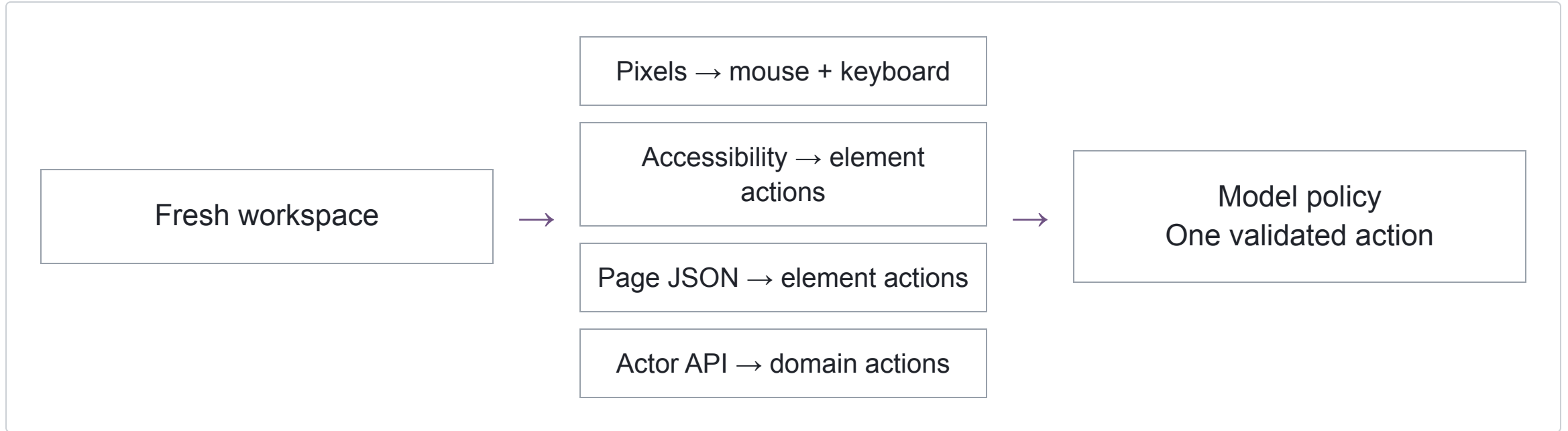
- Runnable web application with persistent, synthetic workspace state.
- 18 tasks covering search, threads, edits, DMs, pins, saved messages and channel details.
- Independent terminal-state evaluation and inspectable agent trajectories.
- Kevin Liu · CUA implementation exercise · October 2026.



Implementation scope and prior work

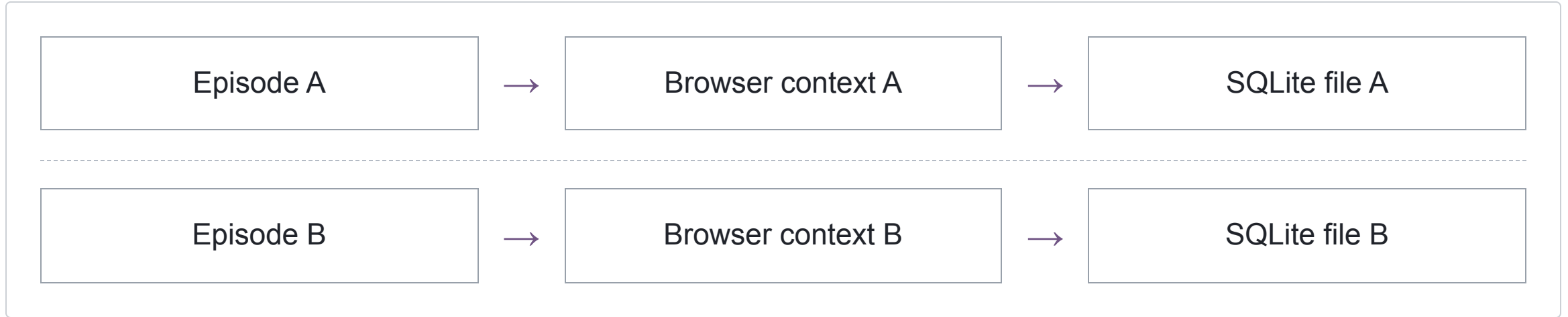
- React UI, deterministic transition engine, SQLite persistence and Playwright runner.
 - Static Slack mockups supply appearance, but not reset, reward or durable outcomes.
 - Cua Slack, env0 and Agent-Diff already address related environments; this is not a novelty claim.
 - Original focused implementation keeps task contracts and failure traces easy to inspect.
-
- MIT for original code; third-party asset restrictions remain separate.
 - Source review, not comparative execution: [prior-work survey and pinned references](#).

Four observation and action interfaces



- The actor never receives grader answers or control credentials.
- API access changes information and action granularity; it is not a computer-use score.
- Optional documentation and bounded history are explicit conditions, not hidden prompt changes.

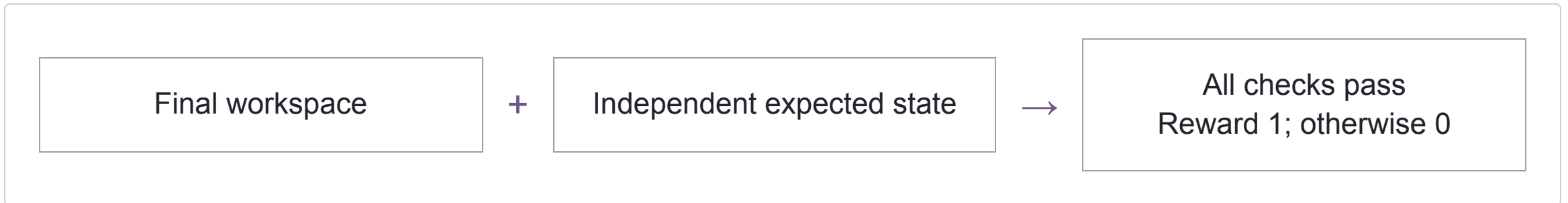
Episode lifecycle and session isolation



- Fresh contexts and private files prevent cross-episode state contamination.
- Transactional writes, revisions and request IDs enforce atomicity and idempotency.
- Actor and control listeners share a process: this is data isolation, not a hostile-code sandbox.

Tasks require correct state, not a “done” response

- Six control tasks; 12 deeper workflows with 3–6 coordinated mutations.
- Decision record: retrieve final approval, update description, pin the root, save the reply and acknowledge in-thread.
- Check content, identity, author, destination, uniqueness and all unchanged workspace state.
- Wrong thread, replacement instead of edit, stale fact or collateral change fails.



- Exact text is deliberate. Fully undone actions may pass; path-level safety is not currently graded.

Verification separates software checks from model runs

- 121 backend/harness checks and 77 browser checks.
- 84 valid states and 2,583 adversarial invalid states for the 12 deep-task graders.
- Scripted UI recipes exercise interaction paths; they do not measure model capability.
- 20 newly attempted real-model episodes; original builder-informed episodes remain labeled separately.

Software tests
Does the app work?

Grader challenges
Does reward mean success?

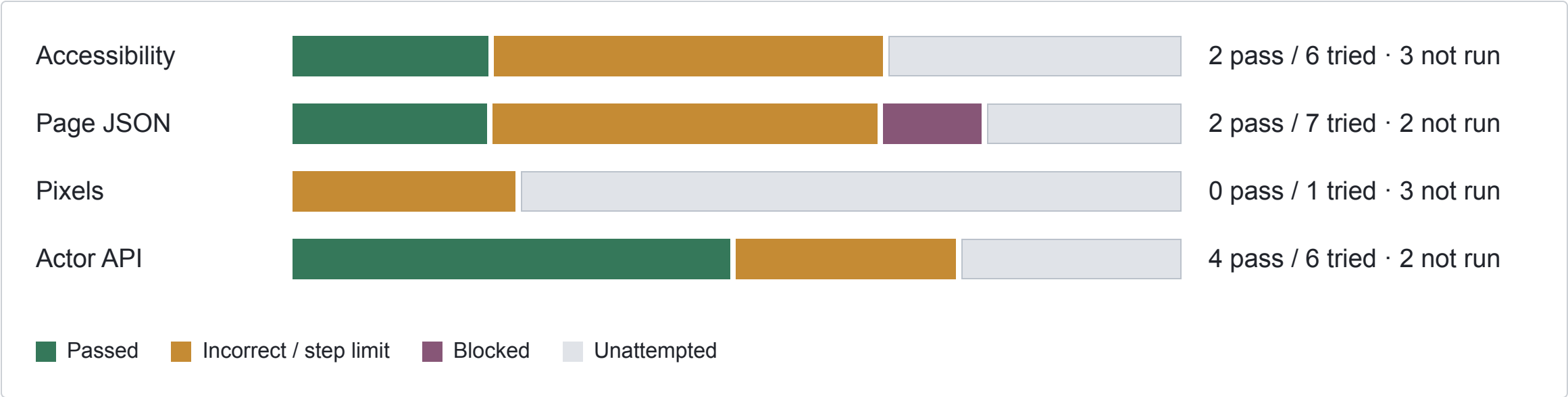
Model episodes
Can the policy do the task?

A bounded, matched development campaign

- GPT-4o mini and GPT-6 Luna through Ramp Router; exact requested/returned model receipts retained.
- 30 planned cells: pilot, matched workflow matrix and separate pixel follow-up.
- Fresh sessions; seed 42; seeded cell order; recent-four history; no supplied guide.
- \$3 ceiling; fixed action, token and time limits; no retry or model substitution.
- One seed and one repetition on public tasks: descriptive evidence, not a model ranking.
- Frozen plan, exact configuration and pilot review.

Observed outcomes, including incomplete coverage

- 20 attempted: 8 passed, 11 incorrect/step-limited, 1 blocked; 10 unattempted.
- 126 requests; \$0.149874 estimated including \$0.001490 reserved for the blocked response.



- Unequal task/model coverage; these totals are not causal interface comparisons. Cost is not an invoice.
- [Complete inventory, exact trajectories and hashes.](#)

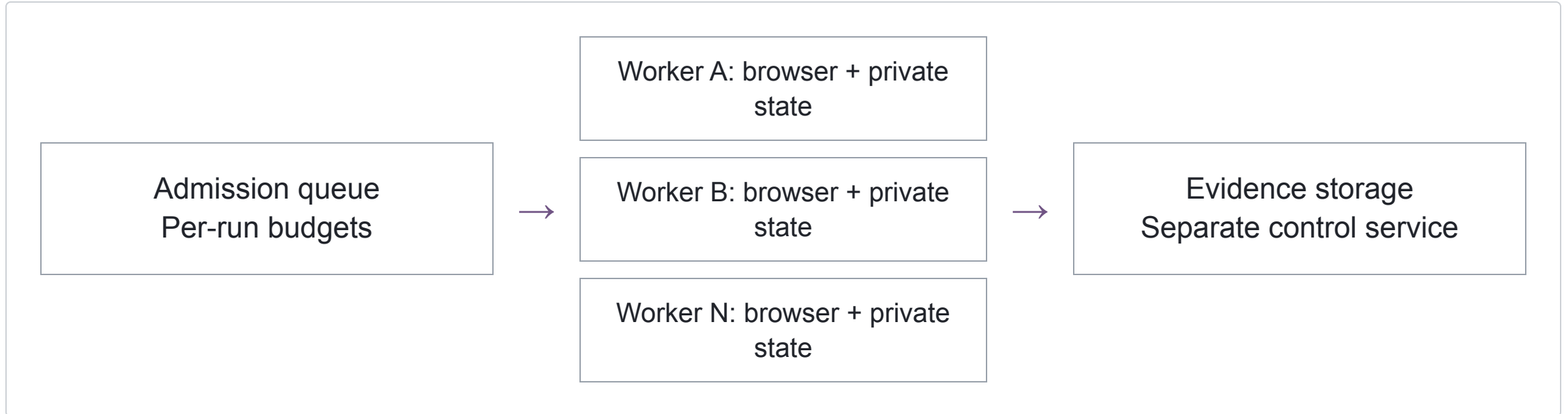
What the trajectories explain

- Thread reply: GPT-6 Luna passed through accessibility, page JSON and API; GPT-4o mini failed those matched cells.
- Decision record: Luna pinned and saved correctly in the UI, but copied the placeholder “DESIGN” instead of the approved fact. API execution passed.
- Pixel pilot: 20 malformed multi-action responses were rejected; no state mutation occurred.
- Stop condition: Luna hit the 1,024-output-token limit. The incomplete response executed no action; remaining cells were not launched.
- The runner retained the incomplete response's reservation conservatively. This was not HTTP 403 or proof of missing provider access.

Speed and resource measurements

- Current local measurement: create p50 17.34 ms; reset p50 1.50 ms; screenshot p50 84.15 ms.
- 100 persisted sessions, 400 reads at concurrency 16, and 10 sequential browser contexts.
- Machine: Apple M5 Max, Node 24; timings include the stated operation only.
- Browser child-process memory, sustained agent concurrency and multi-host capacity are not measured.
- Model calls and visual evidence add latency; API throughput is not agent throughput.
- [Raw samples, machine details and measurement boundaries.](#)

Scaling path and tradeoffs



- Proposed architecture, not a measured deployment at scale.
- Partition synchronous SQLite work; pool bounded browser processes and recycle workers.
- Use containers/VMs and network isolation before allowing shell-capable or hostile policies.
- Measure full worker memory, p95 latency, failure rate and cleanup under a sustained load.

Demonstration and discussion

- [Open the workspace](#): search, reply in a thread, and edit your own message.
 - [Open Replays](#): compare the successful thread reply and the incomplete decision record.
 - Inspect the exact request, action, state change and terminal checks; replay requires no model calls.
 - Discuss: required Slack fidelity, observation contract, security boundary and cost/throughput target.
 - Next gates: successful pixel policies, task-template holdouts, trainer integration and concurrent-worker measurements.
-
- Reset/step/reward are implemented; no RL training or BrowserGym/OpenEnv compatibility is claimed.
 - [Code and setup](#) · [Submission assessment and limitations](#).